



INF2010 – Structures de données et algorithmes

Hiver 2026

TP No. 5

Groupe 2

2435049 – Nicolas Mihali

2420324 – Akhan Mehmet Sozen

Date de la remise : 5 avril 2026

1. Décrivez la logique de votre solution pour la partie 1 et analysez la complexité temporelle et spatiale.

Description de la logique.

Étape 1 : On compare une paire de mots (le mot courant ainsi que celui qui vient après). Pour chaque paire, on compare les caractères position par position. Lorsqu'on trouve que deux caractères sont différents, on crée une arête orientée dans le graphe entre ces deux caractères et donc letter1 vient avant letter2 dans l'alphabet. On s'arrête immédiatement après avoir trouvé la première différence, car c'est la seule information qui nous permet de déduire l'ordre entre les deux mots.

Étape 2 : On itère sur tous les sommets du graphe, si un sommet a un indegré de 0, on le met en premier dans la file.

Étape 3 : On trie en ordre topologique avec DFS. En effet, on commence par extraire les sommets de la file un par un et on les ajoute à l'ordre lexical. Pour chacun, à l'aide de l'algorithme DFS, on décrémente l'indégré de ses voisins et si un voisin a un indégré de 0, il est ajouté à la file. On répète jusqu'à ce que la file soit épuisée.

Analyse de la complexité.

Soit N = nombre de mots dans dictionary, L = longueur moyenne d'un mot, C = nombre total de caractères unique (sommets), E = nombre d'arêtes (relations entre caractères).

Temporelle

Étape 1 : `for (int i = 0; i < dictionary.length - 1; i++)` est $O(N-1)$ et la boucle intérieure est $O(L)$, alors la complexité de l'étape 1 est $O((N-1) * L) = O(N*L)$.

Étape 2 : `for (Vertex<Character> vertex : graph.vertices)` est $O(C)$.

Étape 3 : La boucle intérieure est $O(E)$ et la boucle while va parcourir chaque sommet, comme chaque sommet et arête sont seulement visités une fois, on obtient $O(C+E)$.

Enfin, la complexité totale est $O(N*L + C + E + C) = O(N*L + C + E)$.

Spatiale

Le graphe stocke C sommets et E arêtes. La file peut contenir au plus C sommets et le tableau de résultat stocke aussi C éléments. Donc on a $O(C + E)$.

2. Décrivez la logique de votre solution pour la partie 2 et analysez la complexité temporelle et spatiale.

Description de la logique

Étape 1 : Soit le labyrinthe, une matrice 2D de taille $M * N$. On utilise deux boucles pour itérer sur chaque sur chaque tuile afin de trouve l'entrée. Comme l'entrée et la sortie sont interchangeables, qu'elle sortie on trouve en premier n'importe pas, car nous cherchons seulement la plus petite distance entre celles-ci. Quand une des sorties est trouvée, on sort de la boucle extérieure en utilisant un label.

Étape 2 : On veut trouver le chemin le plus court à l'aide de BFS. On crée un tableau booléen visited de taille $M \times N$ pour éviter de revisiter les cases. On crée aussi une liste de directions (gauche, droite, haut, bas) qui sont comme des arêtes dans un graphe. Une file est initialisée avec la position de départ sous forme d'un tableau de 3 valeurs [row, col, distance], ce qui permet de mettre la distance accumulée directement dans la file. La distance de départ est initialisée à 0.

Étape 3 : Comme dans la file, nous y avons stocker des tableaux contenant une position de rangée, colonne ainsi qu'une distance, on commence par extraire la tête de la file. Si la case courante est une tuile Exit qui est différente du point de départ, on retourne la distance accumulée. Sinon, on explore les 4 voisins. Si un voisin est dans les bornes de la grille et qu'il n'est pas un mur et qu'il n'a pas encore été visité, on le marque comme visité et on l'ajoute à la file avec une distance incrémentée de 1. Si la file se vide sans trouver la sortie, on retourne null.

Analyse de la complexité.

Soit N = nombre de colonnes et M = nombre de lignes.

Temporelle

Étape 1 : Les deux boucles imbriquées itèrent sur chaque tuile de la matrice, donc la complexité est $O(M*N)$.

Étape 3 : La boucle while va parcourir chaque case une fois seulement grâce à visited et pour chaque case on explore au plus 4 voisins, comme chaque case est visitée maximum une seule fois, la complexité est $O(4*M*N) = O(M*N)$.

Enfin, la complexité totale est $O(M*N)$.

Spatiale

Le tableau visited occupe $O(M*N)$ en mémoire et dans le pire case, la file peut contenir $O(M*N)$ éléments, donc la complexité est $O(M*N)$.

3. Faites une rétrospection sur vos apprentissages avec les SIAG durant cette session.

On a beaucoup aimé le fait que les TPs ont un lien direct avec des problèmes d'entrevues réels. Le fait qu'on ait travaillé avec des structures de données sur des cas pratiques nous a permis de beaucoup mieux comprendre la théorie et de savoir comment l'appliquer en vraie vie, cela rend l'apprentissage beaucoup plus motivant.

Le fait d'utiliser des SIAG dans les TPs a renforcé l'idée que ce sont de très bons outils d'apprentissage mais qu'ils font parfois quand même des erreurs, donc quand un SIAG propose une solution, il faut quand même la vérifier pour que celle-ci ne contienne pas d'erreur.

Bonus

The screenshot shows the LeetCode interface for problem 79, "Word Search". The problem description states: "Given an $m \times n$ grid of characters `board` and a string `word`, return `true` if `word` exists in the grid. The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once."

Example 1:

A	B	C	E
S	F	C	S
A	D	E	E

Input: `board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ], word = "ABCCED"`
Output: `true`

Example 2:

A	B	C	E
S	F	C	S
A	D	E	E

The C++ code on the right implements a backtracking solution:

```
class Solution {
public:
    bool exist(vector<vector<char>>& board, string word) {
        int m = board.size();
        int n = board[0].size();

        // On essaie de démarrer la recherche depuis chaque case du tableau
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (backtrack(board, word, i, j, 0)) {
                    return true;
                }
            }
        }
        return false;
    }
};
```

The test results show "Accepted" with a runtime of 0 ms. The test case input is the same as Example 1, and the output is `true`.

Depth-First Search

Submit

Premium

DescriptionAcceptedEditorialSolutionsSubmissions

All Submissions

Accepted88 / 88 testcases passed

ICYCOOLBEANZ submitted at Mar 31, 2026 20:18

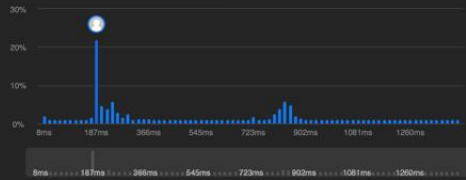
AnalysisSolution

Runtime

191 msBeats: 83.07%

Memory

10.38 MBBeats: 93.03%



8ms167ms366ms545ms723ms902ms1081ms1290ms

0%10%20%30%

Code | C++

```
1 class Solution {
2 public:
3     bool exist(vector<vector<char>>& board, string word) {
4         int m = board.size();
5         int n = board[0].size();
6
7         // On essaie de démarrer la recherche depuis chaque case du tableau
8         for (int i = 0; i < m; i++) {
9             for (int j = 0; j < n; j++) {
10                 if (backtrack(board, word, i, j, 0)) {
11                     return true;
12                 }
13             }
14         }
15         return false;
16     }
```

View more

More challenges

212. Word Search II

Write your notes here

Code

C++

```
1 class Solution {
2 public:
3     bool exist(vector<vector<char>>& board, string word) {
4         int m = board.size();
5         int n = board[0].size();
6
7         // On essaie de démarrer la recherche depuis chaque case du tableau
8         for (int i = 0; i < m; i++) {
9             for (int j = 0; j < n; j++) {
10                 if (backtrack(board, word, i, j, 0)) {
11                     return true;
12                 }
13             }
14         }
15         return false;
16     }
```

Saved

Ln 13, Col 14

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

board =

[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]

word =

"ABCCED"

Output

true

Expected

true